T4-Entrada/Sortida

SISTEMES OPERATIUS
GRAU EN ENGINYERIA INFORMÀTICA

Autors: Professors de Sistemes Operatius del Departament d'Arquitectura de Computadors (UPC- BarcelonaTech)

Llicència



«By Universitat Politècnica de Catalunya - BarcelonaTech (UPC), Any 2025»

https://creativecommons.org/licenses/by-nc-sa/4.0/deed.ca

Índex

- Conceptes bàsics d'E/S
- Dispositius: Virtual, lògic i Físic
- Gestió de l'E/S a Linux
  - Estructures de dades del kernel
  - Crides a sistema bàsiques
  - Exemples
- Relació entre crides a sistema / estructures de dades



CONCEPTES BÀSICS D'E/S

Què és l'E/S?

- **Definició**: transferència d'informació entre un procés i l'exterior
  - Entrada de dades: de l'exterior al procés
  - Sortida de dades: del procés a l'exterior

  (sempre des del punt de vista del procés)

- De fet, bàsicament, els processos realitzen càlcul i E/S
- Molts cops, inclús, l'E/S és la tasca principal d'un procés:
  p. ex. navegació web, intèrpret de comandes, processador de texts

- **Gestió de l'E/S**: administrar el funcionament dels dispositius d'E/S (perifèrics) per a un ús **correcte**, **compartit** i **eficient** dels recursos

Dispositius d'E/S







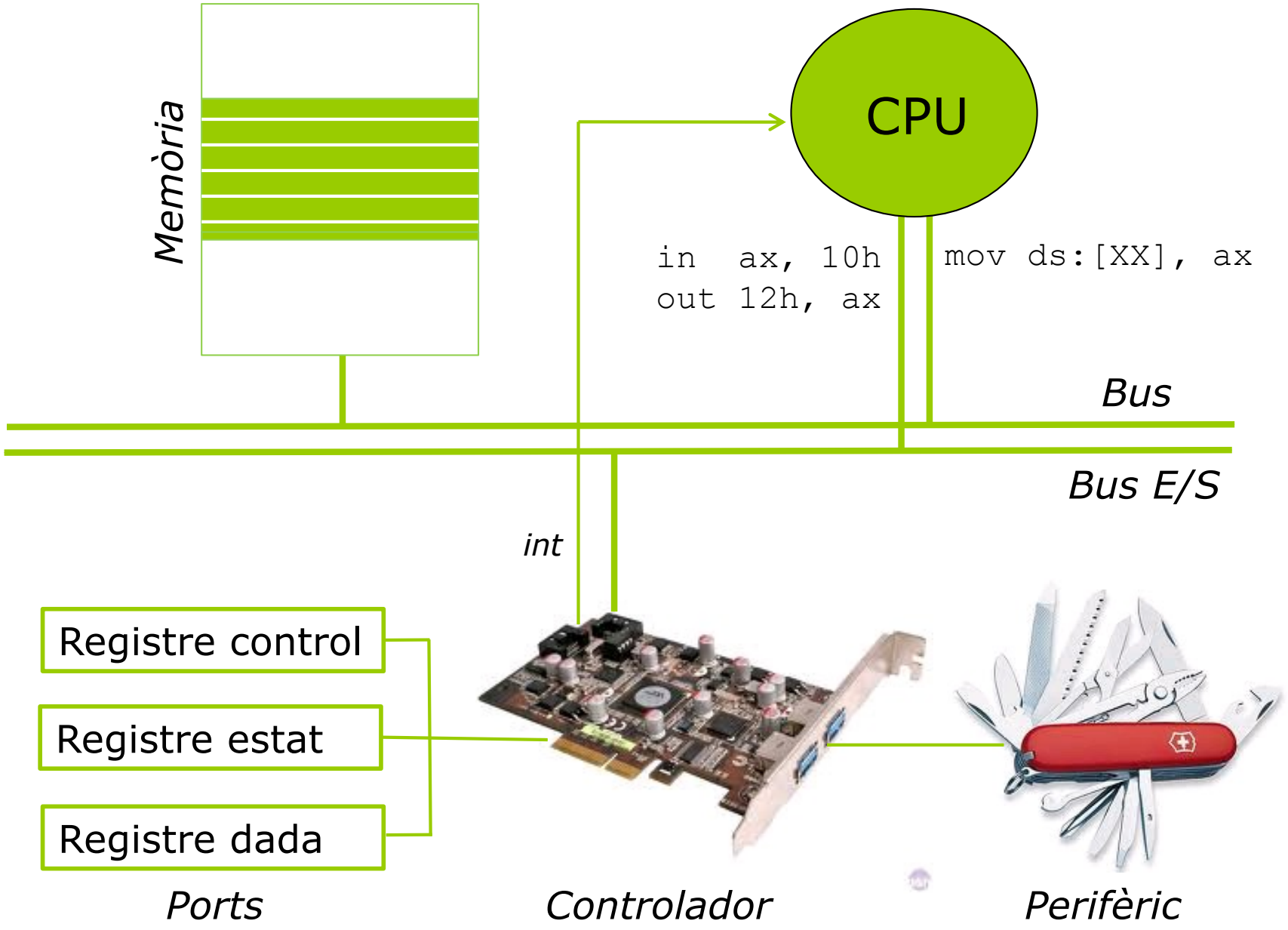




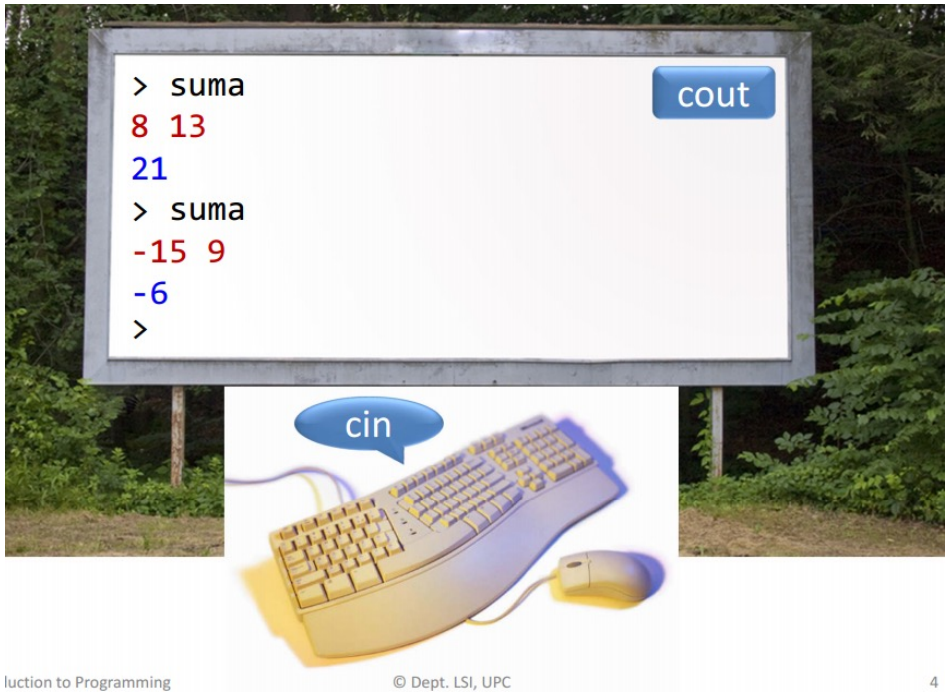




Visió HW: Accés als dispositius d'E/S

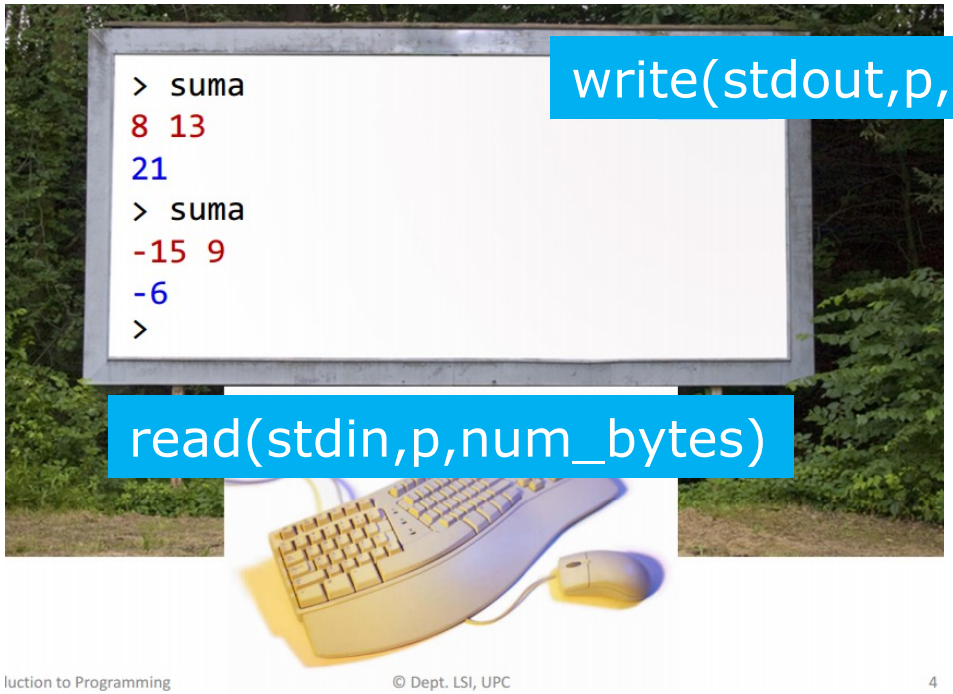


Visió usuari (fins ara) (PRO1)



Entrada: **cin**. Llegeix les dades i les processa per adaptar-les al tipus de dades
Sortida: **cout**. Processa les dades i les escriu

En aquest curs veurem que hi ha al mig



- No serà tan fàcil com a C o C++
- La conversió de tipus de dades és responsabilitat de l'usuari (és el que fan les llibreries de C o C++)

DISPOSITIUS:VIRTUAL, LÒGIC, FÍSIC



Tipus de dispositius

- Dispositius d'interacció amb l'usuari (pantalla, teclat, mouse), emmagatzemament (disc dur, DVD, pen drive), transmissió (mòdem, xarxa amb cable, xarxa sense cable) o inclús d'altres més especialitzats (controlador d'un avió, sensors, robots) … moltíssima varietat!
- Classificació segons:
  - Tipus de dispositiu: lògic, físic, de xarxa
  - Velocitat d'accés: teclat vs disc dur
  - Flux d'accés: mouse (byte) vs DVD (bloc)
  - Exclusivitat d'accés: disc (compartit) vs impressora (dedicat)
  - Altres…
- Dilema: estandardització vs nous tipus de dispositius

CONCEPTE: Independència dels dispositius

Independència: principis de disseny

- L'objectiu és que els processos (codi principalment) sigui independent del dispositiu que s'està accedint
- Operacions d'E/S **uniformes**
  - Accés a tots els dispositius mitjançant les **mateixes crides al sistema**
  - Augmenta la simplicitat i la portabilitat dels processos d'usuari
- Utilitzant **dispositius virtuals**
  - El procés no especifica concretament sobre quin dispositiu treballa, sinó que utilitza uns identificadors i hi ha una traducció posterior
- Amb possibilitat de **redireccionament**
  - El sistema operatiu permet a un procés canviar l'assignació dels seus dispositius virtuals

```
% programa < disp1 > disp2
```



Independència: principis de disseny

- Per això, habitualment, disseny en tres nivells: **virtual**, **lògic** i **físic**
- El primer nivell ens aporta independència, el **procés treballa amb dispositius virtuals** i no necessita saber que hi ha al darrere
- **El segon nivell, ens aporta compartició de dispositius.** Diferents accessos concurrents al mateix dispositiu.
- D'aquesta manera es poden escriure programes que realitzin E/S sobre dispositius (virtuals) sense especificar quins (lògic).
- En el moment d'executar el programa es determina dinàmicament sobre quins dispositius es treballa
  - Pot ser un paràmetre del programa, pot ser "heredat" del seu pare,...
- El tercer nivell separa les operacions (software) de la implementació. Aquest codi és molt baix nivell, moltes vegades en assemblador

Dispositiu virtual

- **Nivell virtual**: Aïlla l'usuari de la complexitat de gestió dels dispositius físics
  - Estableix correspondència entre nom simbòlic (nom de fitxer) i l'aplicació d'usuari, a través d'un dispositiu virtual
    - Un nom simbòlic és la representació al sistema d'un dispositiu
      - `/dev/dispX` o bé `.../dispX un nom de fitxer`
    - Un dispositiu virtual representa un dispositiu en ús d'un procés
      - Dispositiu virtual = canal = descriptor de fitxer. És un **nombre enter**
      - Els processos tenen 3 canals estàndard:
        - Entrada estàndard canal 0 (stdin)
        - Sortida estàndard canal 1 (stdout)
        - Sortida error estàndard canal 2 (stderr)



  - Les crides a sistema de transferència de dades utilitzen com identificador el dispositiu virtual

Dispositiu lógic

- **Nivell lògic**:
  - Estableix correspondència entre dispositiu virtual i dispositiu (físic?)
  - Gestiona dispositius que poden tenir, o no, representació física
    - P. ex. Disc virtual (sobre memòria), dispositiu nul
  - Manipula blocs de dades de mida independent
  - Proporciona una interfície uniforme al nivell físic
  - Ofereix compartició (accés concurrent) als dispositius físics que representen (si hi ha)
  - En aquest nivell es tenen en compte els permisos, etc
  - A Linux s'identifiquen amb un nom de fitxer



Dispositiu físic

- **Nivell físic**: Implementa a baix nivell les operacions de nivell lògic
  - Tradueix paràmetres del nivell lògic a paràmetres concrets
    - P. ex. En un disc, traduir posició L/E a cilindre, cara, pista i sector
  - Inicialitza dispositiu. Comprova si lliure; altrament posa petició a cua
  - Realitza la programació de l'operació demanada
    - Pot incloure examinar l'estat, posar motors en marxa (disc), …
  - Espera, o no, a la finalització de l'operació
  - Retorna els resultats o informa sobre algun eventual error
  - A Linux, un dispositiu físic s'identifica amb tres paràmetres:
    - Tipus: **Block/Character**
    - Amb dos nombres: major/minor
      - **Major**: Indica el tipus de dispositiu
      - **Minor**: instància concreta respecte al major



Device drivers

- Per oferir independència, es defineix el conjunt d'operacions que ofereixen els dispositius
  - És un superconjunt de totes les operacions que es poden oferir per accedir a un dispositiu físic
  - No tots els dispositius ofereixen totes les operacions
  - Quan es tradueix de disp. Virtual a lògic i físic , d'aquesta manera està definit quines operacions corresponen



GESTIÓ DE L'E/S A LINUX 

Tipus de fitxers a Linux



- A Linux, tots els dispositius s'identifiquen amb un fitxer (que pot ser de diferents tipus)
  - block device
  - character device
  - Directory
  - FIFO/pipe
  - Symlink
  - **regular file → Son els fitxers de dades o "ordinary files" o fitxers "normals"**
  - Socket
- Anomenarem fitxers "especials" a qualsevol fitxer que no sigui un fitxer de dades: pipes, links, etc.

Creació de fitxers a Linux



- La majoria de tipus de fitxers es poden crear amb la crida a sistema/comanda mknod
  - Excepte directoris i soft links
  - Al laboratori utilitzarem la comanda, que **no serveix per crear fitxers de dades**
- mknod nom_fitxer TIPUS major minor
  - TIPUS = c → Dispositiu de caràcters
  - TIPUS = b → Dispositiu de blocs
  - TTIPUS = p → Pipe (no cal posar major/minor)

Tipus de Fitxers



- Alguns exemples:

dev name	type	major	minor	description
`/dev/fd0`	b	2	0	floppy disk
`/dev/hda`	b	3	0	first IDE disk
`/dev/hda2`	b	3	2	second primary partition of first IDE disk
`/dev/hdb`	b	3	64	second IDE disk
`/dev/tty0`	c	3	0	terminal
`/dev/null`	c	1	3	null device

ESTRUCTURES DE DADES DEL KERNEL



Estructures de dades del kernel: Inode



- Estructura de dades que conté tota la informació relativa a un fitxer
  - mida
  - tipus
  - proteccions
  - propietari i grup
  - Data de l'últim accés, modificació i creació
  - Nombre d'enllaços a l'inode (nombre de noms de fitxers que apunten a…)
  - índexs a les dades (indexació multinivell)
- Està tota la informació d'un fitxer excepte el nom, que està separat
- S'emmagatzema a disc, però hi ha una copia a memòria per optimitzar el temps d'accés

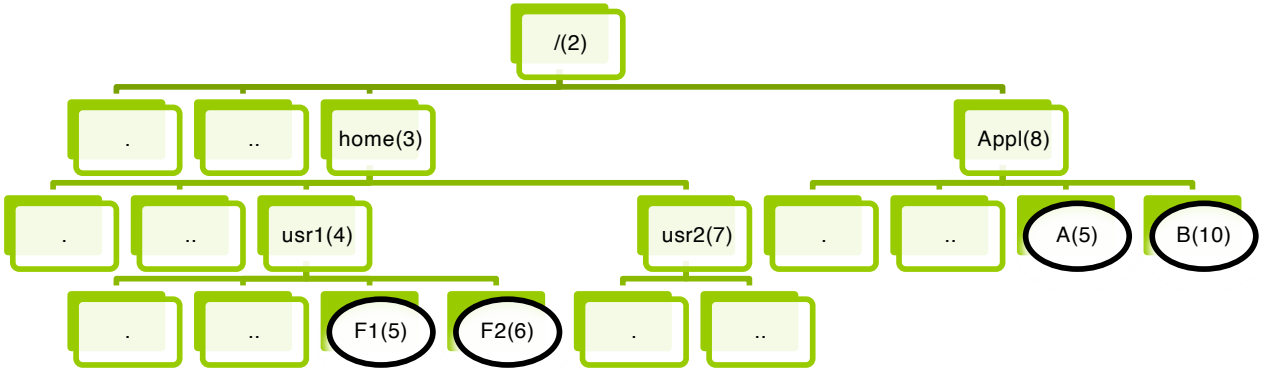
Visió interna de directoris



- El contingut d'un directori permet relacionar el nom del fitxer amb el seu número d'inode

Nom	inode
.	2
..	2
home	3
Appl	8

Directori /



Cas especial

Nom	inode
.	4
..	3
F1	5
F2	6

Directori /home/usr1

```
# ln /home/usr1/F1 /Appl/A
# ls -l /Appl/A
-rw-r--r--  2 ken  staff  5 16 juny  18:00 Appl/A
# ln -s /home/usr1/F2 /Appl/B
# ls -l /Appl/B
lrwxr-xr-x  1 ken  staff  13 16 juny  18:01 Appl/B
-> home/usr1/F2
```

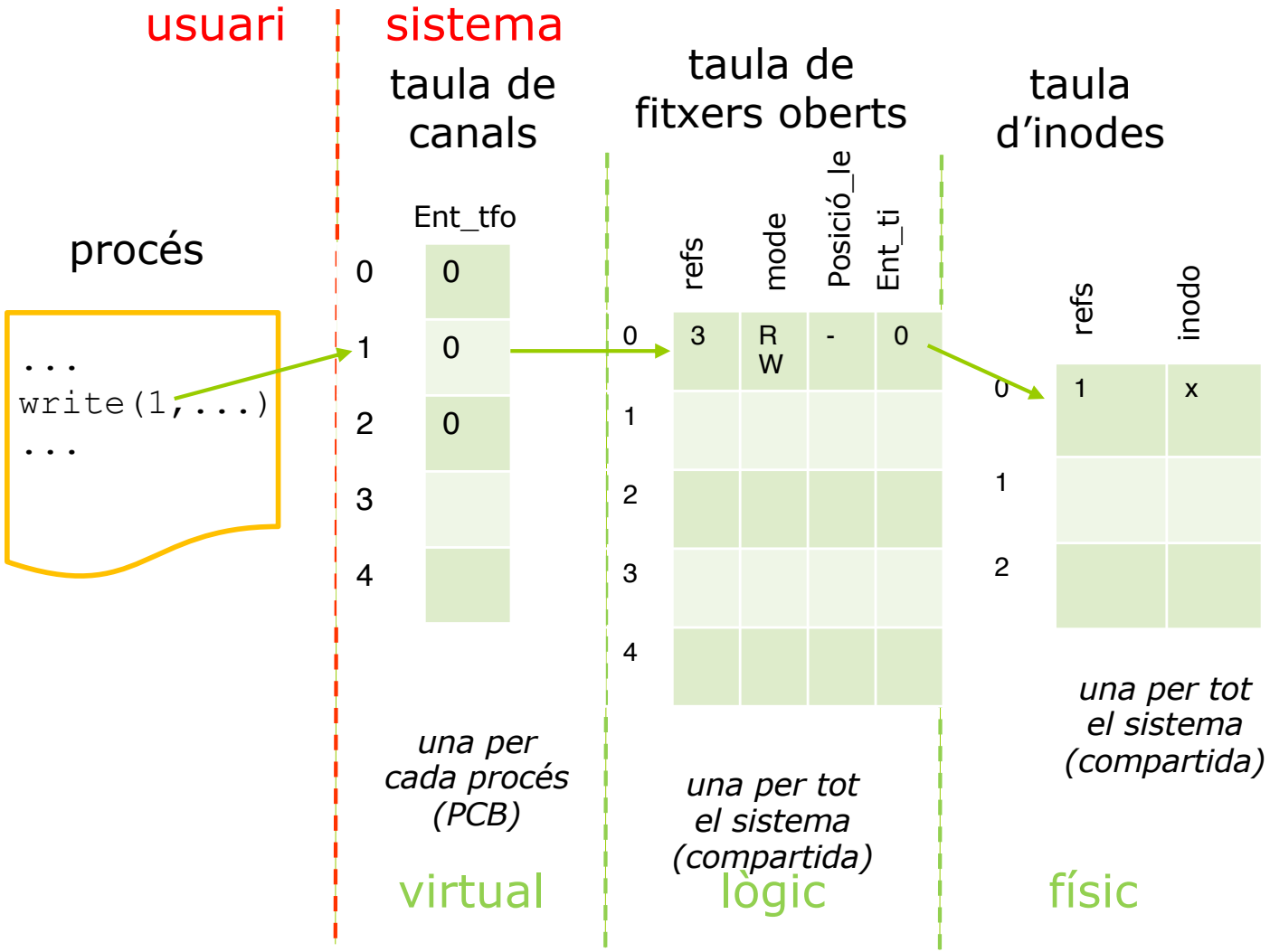
Estructures de dades de kernel



- Per procés
  - Taula de canals (TC): per cada procés (es guarda al PCB)
    - Indica a quins fitxers estem accedint.
    - S'accedeix al fitxer mitjançant el canal, que és un **índex a la TC**
    - El canal és el disp. Virtual
    - Un canal referència a una entrada de la taula de fitxers oberts (TFO)
    - Camps que assumiren: `num_entrada_TFO`
- Global:
  - Taula de fitxers oberts (TFO):
    - Gestiona els fitxers en ús en tot el sistema
    - Poden haver entrades compartides entre més d'un procés i per varies entrades del mateix procés
    - Una entrada de la TFO referencia a una entrada de la TI.
    - Camps que assumirem: `num_referencies, mode , punter_le, num_entrada_ti`
  - Taula d'inodes (TI):
    - Conté informació sobre cada objecte físic obert, incloent les rutines del DD
    - És una copia en memòria de les dades que tenim al disc (es copia a memòria per eficiència)
    - Camps que assumirem: `num_referencies, dades_inode`

Estructures de dades de kernel





CRIDES A SISTEMA BÀSIQUES



Ops bloquejants i no bloquejants (1)

- Algunes operacions poden ser costoses en termes de temps, el procés no pot estar consumint CPU sense fer res → El SO bloqueja el procés (RUN→BLOCKED)
- Operació d'E/S **bloquejant**: Un procés realitza una transferència d'N bytes i s'espera a que acabi. Retorna el nombre de bytes que s'han transferit
  - Si les dades es poden disposar al moment (encara que potser no totes), es realitza la transferència i el procés retorna immediatament
  - Si les dades no estan disponibles, **el procés es bloqueja**
    1. Canvi d'estat de RUN a BLOCKED
       1. Deixa la CPU y passa a una cua de processos que estan esperant.
    2. Es posa en execució el primer procés de la cua de preparats (Si Round Robin)
    3. Quan arriben les dades es produeix una interrupció
       - La RSI recull la dada i posa el procés a la cua de preparats (Si Round Robin)
    4. Quan li toqui el torn, el procés es posarà novament en execució



Ops bloquejants i no bloquejants (2)

- Operació d'E/S **no bloquejant**
  - El procés sol·licita una transferència, disposa les dades que pugui en aquell moment, i retorna immediatament tant si hi ha dades com no
  - Retorna el nombre de dades transferides

Operacions bàsiques d'E/S

Crida a sistema	Descripció
open	Donat un nom de fitxer i un mode d'accés, retorna el nombre de canal per poder accedir
read	**Llegeix N bytes d'un dispositiu** (identificat amb un nombre de canal) i ho guarda a memòria
write	Llegeix N bytes de memòria i els **escriu al dispositiu** indicat (amb un nombre de canal)
close	Allibera el canal indicat i el deixa lliure per ser reutilitzat.
dup/dup2	Duplica un canal. El nou canal fa referencia a la mateixa entrada de la TFO que el canal duplicat.
pipe	Crea un dispositiu tipus pipe llesta per ser utilitzada per comunicar processos
mmap	Mapeja N bytes d'un dispositiu (identificat amb un nombre de canal) a memòria, alineat a pàgina
lseek	Mou la posició actual de L/E (per fitxers de dades)

Les crides open, read i write poden bloquejar el procés



Open

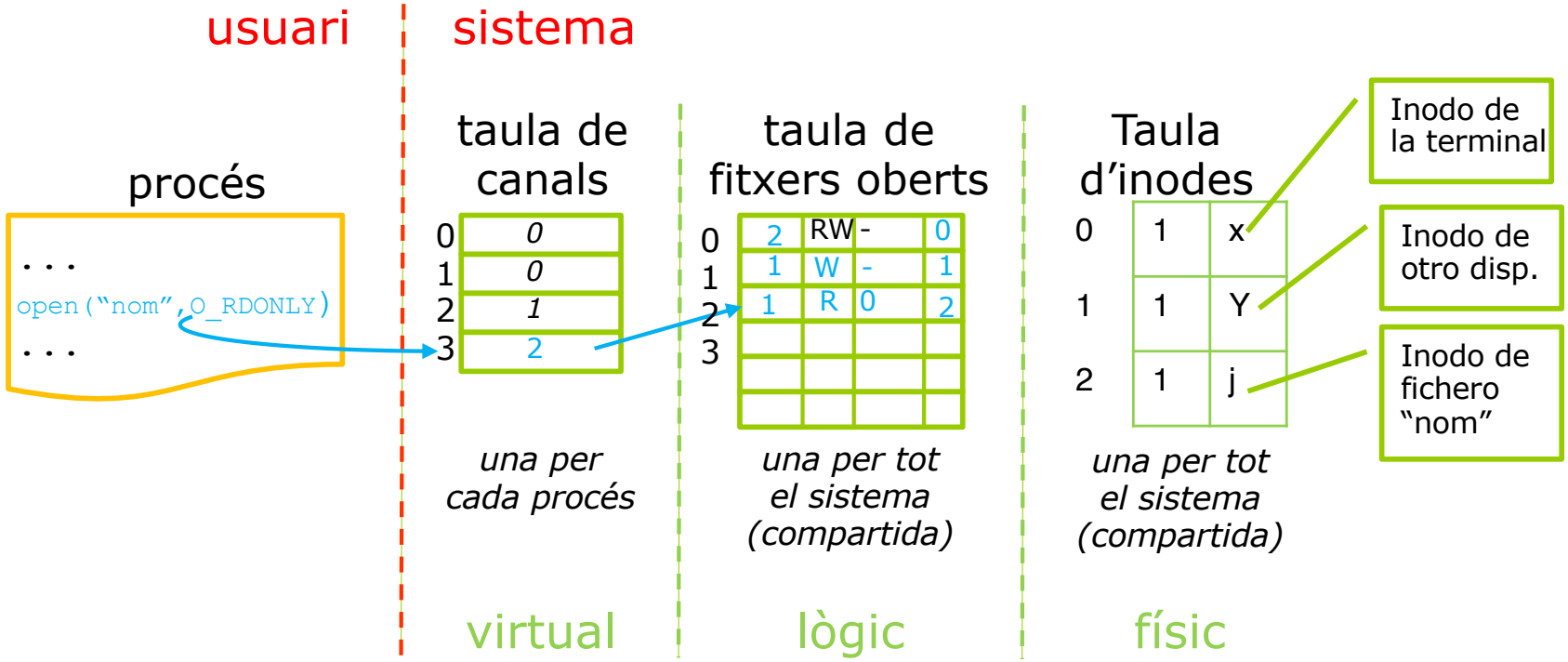
- Per tant, com s'associa un nom a un dispositiu virtual?
- `fd = open(nom, access_mode[,permision_flags]);`
  - Vincula nom de fitxer a dispositiu virtual (descriptor de fitxer o canal)
    - A més, permet fer un sol cop les comprovacions de proteccions. Un cop verificat, ja es pot executar *read/write* múltiples cops però ja no es torna a comprovar
  - Se li passa el **nom** del dispositiu i retorna el dispositiu virtual (*fd: file descriptor* o canal) a través del qual es podran realitzar les operacions de lectura/escriptura ,etc
  - El **access_mode** indica el tipus d'accés. Sempre ha d'anar un d'aquests tres com a mínim :
    - O_RDONLY (lectura)
    - O_WRONLY (escriptura)
    - O_RDWR (lectura i escriptura)

Open: Creació

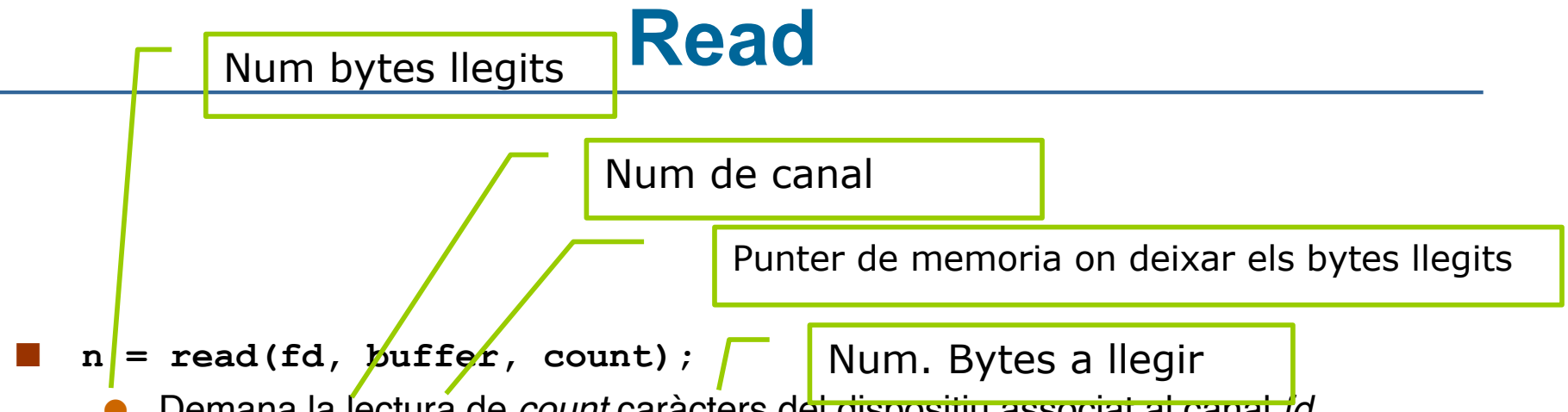
- Els fitxers especials han d'existir abans de poder accedir
- S'han d'especificar els **`permision_flags`**
  - És una OR (|) de: S_IRWXU, S_IRUSR, S_IWUSR, etc
- Els fitxers de dades es poden crear a la vegada que fem l'open:
  - En aquest cas hem d'afegir el flag O_CREAT (amb una OR de bits) a l'acces_mode
- No hi ha una crida a sistema per eliminar dades d'un fitxer parcialment, només les podem esborrar totes. Si volem truncar en contingut d'un fitxer afegirem el flag O_TRUNC a l'acces_mode
  - Ex1: open("X",O_RDWRIO_CREAT, S_IRUSRIS_IWUSR) → Si el fitxer no existia el crea, si existia no té efecte
  - Ex2: open("X",O_RDWRIO_CREATIO_TRUNC, S_IRWXU)→ Si el fitxer no existia el crea, si existia s'alliberen les seves dades y es posa el tamany a 0 bytes.

Open: Estructures de dades

- Open (cont): Efecte sobre les estructures de dades internes del sistema
  - Ocupa un canal lliure de la TC**. <u>Sempre serà el primer disponible</u>**
  - **<u>Ocupa una nova entrada de la TFO: Posició L/E=0</u>**
  - Associa aquestes estructures al DD corresponent (major del nom simbòlic). Pot passar que diferents entrades de la TFO apuntin al mateix DD

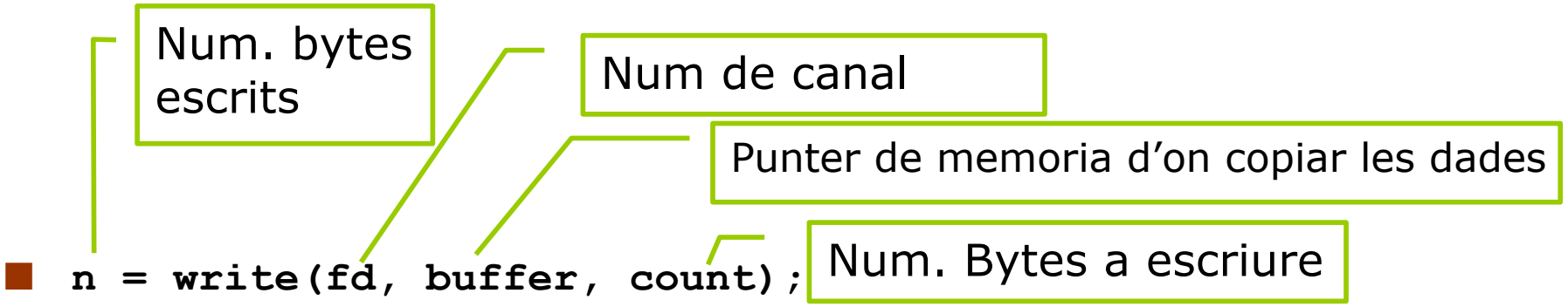


Read



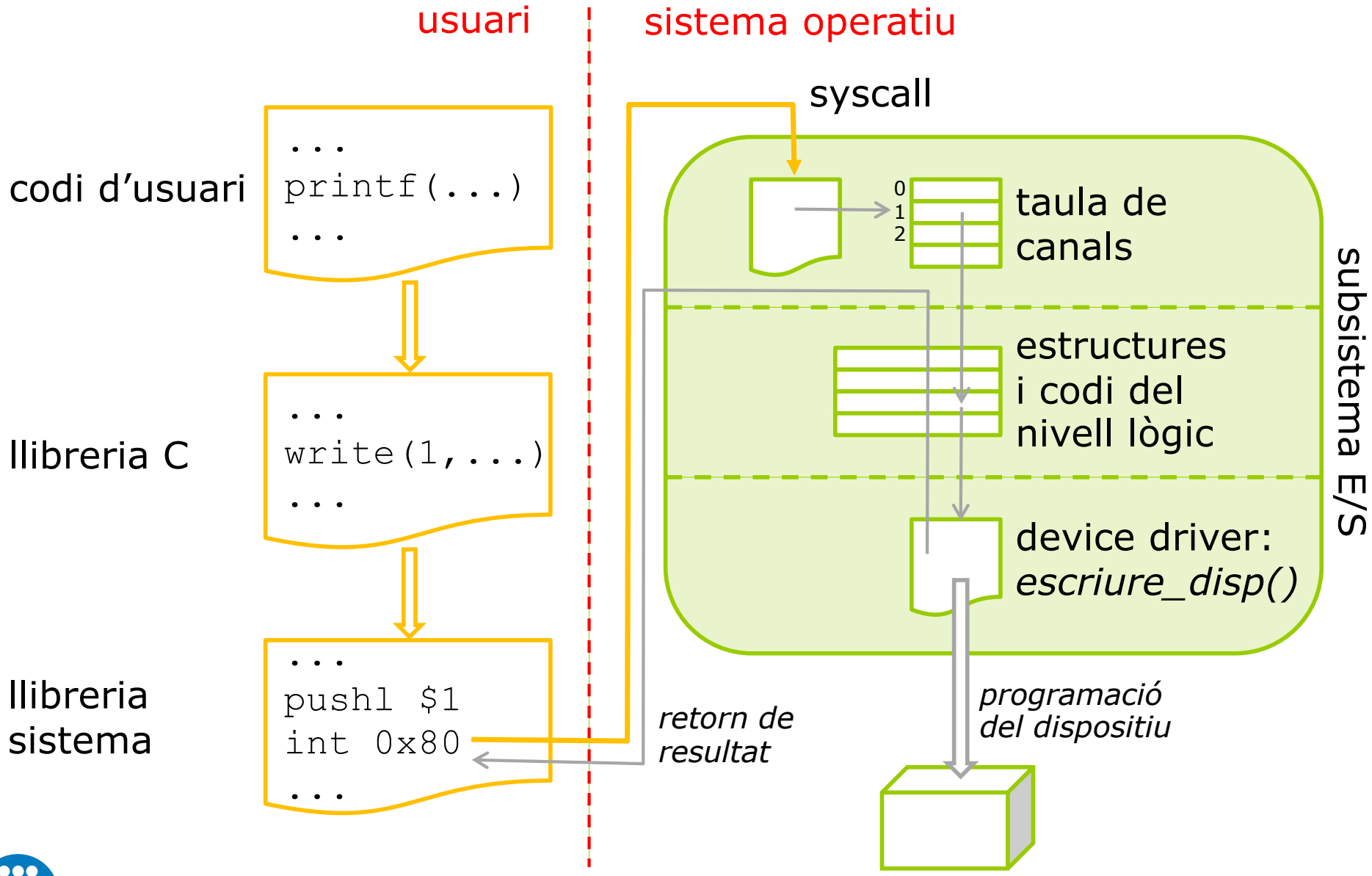
- `n = read(fd, buffer, count);`
  - Demana la lectura de *count* caràcters del dispositiu associat al canal *fd*
    - Si n'hi ha *count*, o més, disponibles, en llegirà *count*
    - Si n'hi ha menys, llegirà els que hi hagi
    - Si no n'hi ha cap, dependrà del funcionament del dispositiu
      - Es pot bloquejar, esperant a que n'hi hagi
      - Pot retornar amb cap caràcter llegit
    - Quan *EOF*, l'operació retorna sense cap caràcter llegit
      - El significat d'*EOF* depèn del dispositiu
  - Retorna *n*, el nombre de caràcters llegits
  - La posició de l/e s'avança **automàticament (ho fa el kernel a la rutina del read)** tantes posicions com bytes llegits
    - Posicio_l/e=posicio_l/e+num_bytes_llegits

Write

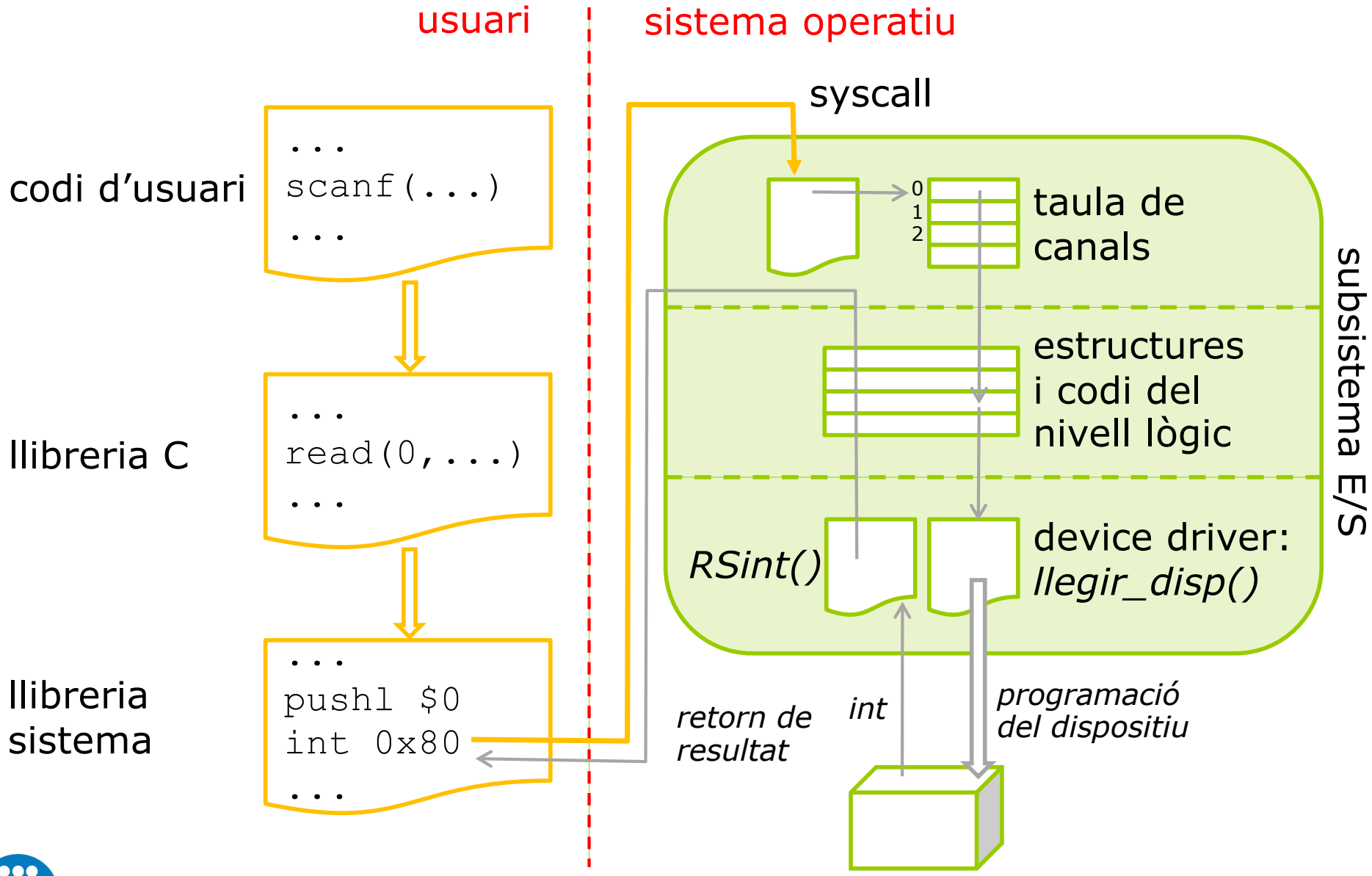


- `n = write(fd, buffer, count);`
  - Demana l'escriptura de *count* caràcters al dispositiu associat al canal *fd*
    - Si hi ha espai per *count*, o més, n'escriurà *count*
    - Si n'hi ha menys, escriurà els que hi càpiguen
    - Si no hi ha espai, dependrà del funcionament del dispositiu
      - Es pot bloquejar, esperant a que hi hagi espai
      - Pot retornar amb cap caràcter escrit
  - Retorna *n*, el nombre de caràcters escrits
  - La posició de l/e s'avança automàticament (ho fa el kernel a la rutina del write) tantes posicions com bytes escrits
    - Posició_l/e=posició_l/e + num_bytes_escrits

Exemple: Escriptura a un dispositiu



Exemple: Lectura d'un dispositiu



Dup/dup2/close



- `newfd = dup(fd);`
  - Duplica un canal
  - Ocupa el primer canal lliure, que ara contindrà una còpia de la informació del canal original *fd*
  - Retorna *newfd*, l'identificador del nou canal



- `newfd = dup2(fd, newfd);`
  - Igual que *dup*, però el canal duplicat és forçosament *newfd*
  - Si newfd era un canal vàlid, prèviament es tanca



- `close(fd);`
  - Allibera el canal *fd* i les estructures associades dels nivells inferiors
  - Cal tenir en compte que diferents canals poden apuntar a la mateixa entrada de la TFO (p. ex. si *dup*), per tant, tancar un canal significaria decrementar -1 el comptador de referències a aquella entrada
  - El mateix passa amb els apuntadors a la llista d'inodes

pipe

- `pipe(fd_vector); // Dispositiu de comunicacions FIFO`



  - Crea una *pipe* **sense** nom. Retorna 2 canals, un de lectura i un altre d'escriptura
  - No utilitza cap nom del VFS (*pipe* **sense nom**) i, per tant, no executa la crida al sistema *open*
  - Només podrà ser usada per comunicar el procés que la crea i qualsevol descendent (directe o indirecte) d'aquest (pq heretaran els canals)
  - Per crear una *pipe* **amb** nom, cal fer: *mknod + open*
  - Internament: També crea 2 entrades a la taula de fitxers oberts (un de lectura i un d'escriptura) i una entrada temporal a la taula d'inodes.

pipe

- Utilització
  - Dispositiu per comunicar processos
  - És bidireccional però , idealment cada procés l'utilitza en una única direcció, es aquest cas, el kernel gestiona la sincronització
- Dispositiu bloquejant:
  - Lectura: Es bloqueja el procés fins que hi hagi dades a la *pipe*, tot i que no necessàriament la quantitat de dades demanades.
    - Quan no hi ha cap procés que pugui escriure i *pipe* buida, provoca EOF (*return==0) → Si hi ha processos bloquejats, es desbloquegen*
  - Escriptura: El procés escriu, a no ser que la *pipe* estigui plena. En aquest cas es bloqueja fins que es buidi.
    - Quan no hi ha cap procés que pugui llegir el procés rep una excepció del tipus *SIGPIPE → Si hi ha processos bloquejats, es desbloquegen*
  - Cal tancar els canals que no s'hagin de fer servir! Altrament bloqueig
- Estructures de dades
  - Es creen dues entrades a la taula de canals (R/W)
  - Es creen dues entrades a la Taula FO (R/W)
  - Es crea una entrada temporal a la Taula d'Inodes

mmap

- **mmap (tema 3):** per a reservar una regió inicialitzada amb el contingut d'un fitxer

void *mmap(void *addr, int nbytes, int permisos, int flags, int fd, int offset)

  - Paràmetre *fd* canal associat a la font de les dades
  - Paràmetre *offset* posició inicial del fitxer que es vol mapejar (si val 0 es mapeja des de l'inici)
  - Paràmetre *flags* pot contenir MAP_SHARED o MAP_PRIVATE
    - Si MAP_SHARED els canvis que es facin a la regió de memòria es reflectiran al fitxer
  - mmap no accedeix al fitxer, només a l'inode per tenir les característiques (mida, permisos…)
  - A l'accedir a la regió reservada (com a qualsevol altra regió de memòria) s'estarà accedint al contingut del fitxer
- **munmap**: allibera una regió reservada amb mmap

  int munmap(void *addr, int nbytes)



lseek

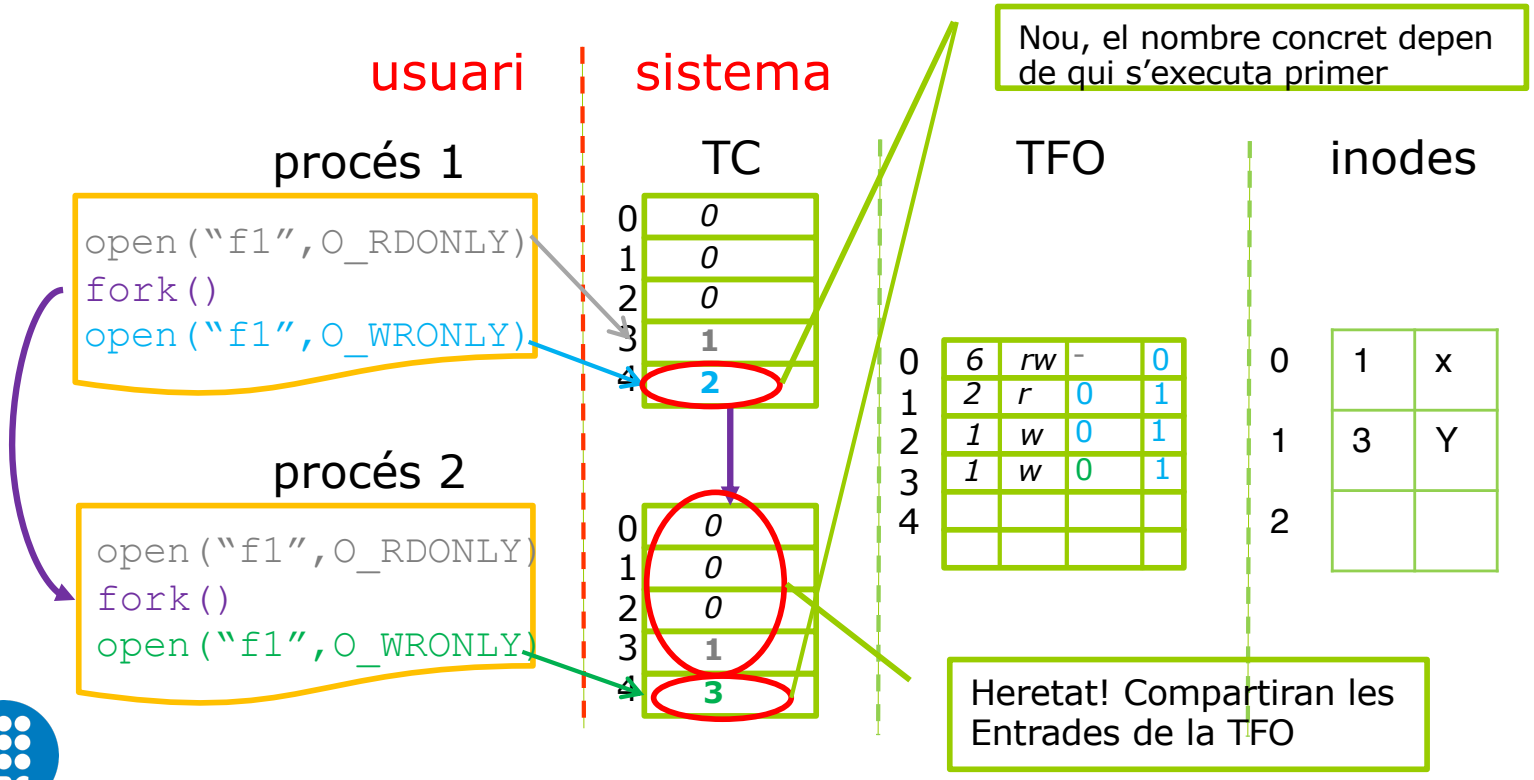
- La posició de l/e es modifica manualment per l'usuari. Permet fer accessos directes a posicions concretes de fitxers de dades (o fitxers de dispositius que ofereixin accés seqüencial)
  - S'inicialitza a 0 a l' open
  - S' incrementa automàticament al fer read/write
  - El podem moure manualment amb la crida lseek

- `nova_posicio=lseek(fd, desplaçament, relatiu_a)`
  - SEEK_SET: posicio_l/e=desplaçament
  - SEEK_CUR: posicio_l/e=posicio_l/e+desplaçament
  - SEEK_END: posicio_l/e=file_size+desplaçament
  - "desplaçament" pot ser negatiu

E/S i execució concurrent (1)



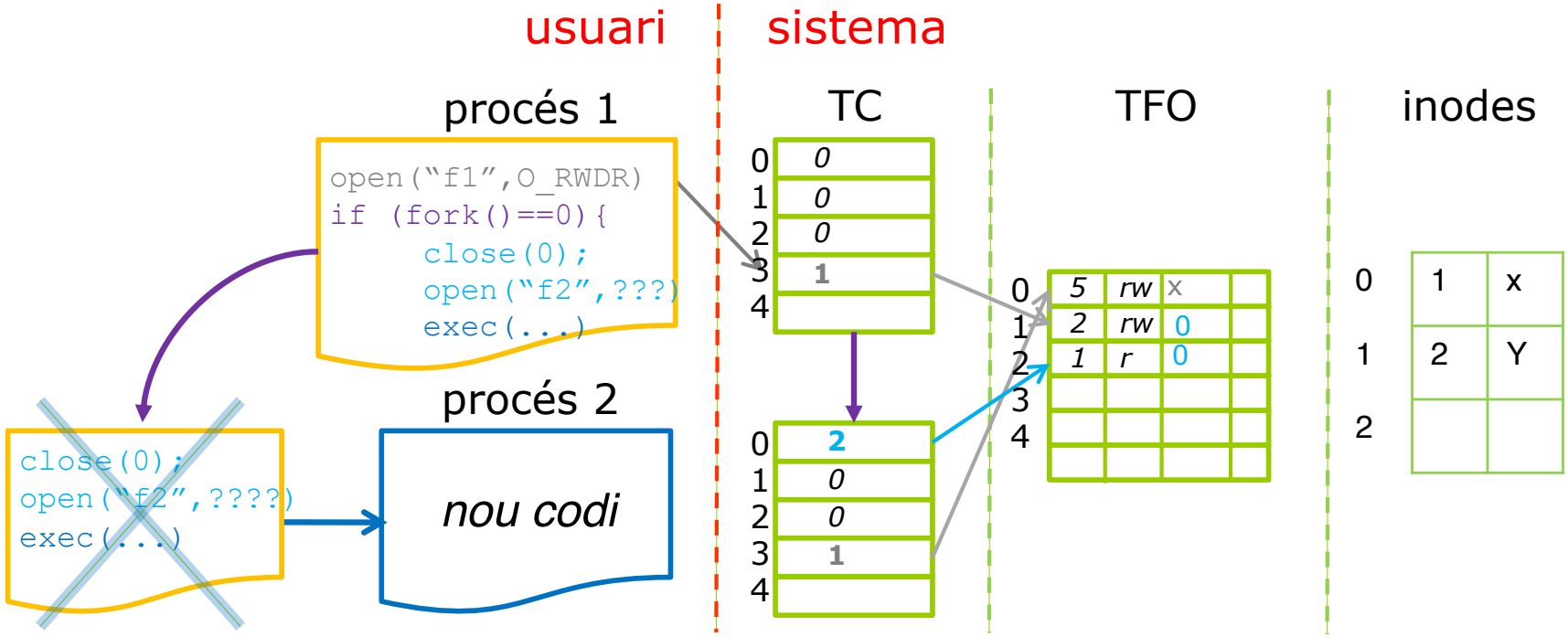
- E/S i *fork*
  - El procés fill hereta una **còpia** de la taula de canals del pare
    - Totes les entrades obertes apunten al mateix lloc de la TFO
  - Permet **compartir** l'accés als dispositius oberts abans del *fork*
  - Els següents *open* ja seran independents



E/S i execució concurrent (2)



- E/S i *exec*
  - La nova imatge **manté** les estructures internes d'E/S del procés
  - El fet d'executar *fork+exec* permet realitzar redireccions abans de canviar la imatge



E/S i execució concurrent (3)



- Si un procés està bloquejat en una operació d'E/S i l'operació és interrompuda per un signal podem tenir dos comportaments diferents:
  - Després de tractar el signal, l'operació interrompuda es reinicia (el procés continuarà bloquejat a l'operació)
  - Després de tractar el signal, l'operació retorna error i la `errno` pren per valor `EINTR`
- El comportament depèn de:
  - La gestió associada al signal:
    - si flag SA_RESTART a la sigaction → es reinicia l'operació
    - en cas contrari → l'operació retorna error
  - L'operació que s'està fent (per exemple, mai es reinicien operacions sobre sockets, operacions per a esperar signals, etc.)
- Exemple de com protegir la crida al sistema segons el model de comportament:

```
while ( (n = read(...)) == -1 && errno == EINTR );
```

Gestió de les característiques dels dispositius



- Encara que les crides a sistema son uniformes, els dispositius no ho son
- Existeixen crides a sistema per modificar característiques específiques dels dispositius lògics i virtuals.
- Dispositiu lògic
  - `ioctl(fd, cmd [, ptr]);`
- Dispositiu virtual
  - `fcntl(fd, cmd, [, args]);`
- Els paràmetres son molt genèrics per oferir flexibilitat

Característiques terminal



- Terminal
  - El sistema manté, per cada terminal, un buffer intern per guardar els caràcters teclejats. Això permet esborrar caràcters abans de ser tractats
  - Posix defineix una funcionalitat particular a certs caràcters especials:
    - ^H: esborrar un caràcter
    - ^U: esborrar una línia
    - ^D: EOF o final de fitxer (final de l'entrada)
  - Pot tenir també un buffer d'escriptura per cada terminal i implementar certes funcions especials, del tipus "pujar N línies", "avançar esquerra"
  - Cada controlador pot implementar el terminal tan complicat com vulgui. P.ex. que es pugui modificar text del mig de la línia
  - Canonical/No canonical: Pre-processat o no dels caràcters abans d'enviar-los al procés (afecta a la lectura)

Característiques terminal



- Terminal: Funcionament (canonical ja que és el cas per defecte)
  - Lectura
    - **El buffer guarda caràcters fins que es pitja el CR**
    - Si hi ha procés bloquejat esperant caràcters, li dona els que pugui
    - Altrament ho guarda; quan un procés en demana li dona directament
    - ^D provoca que la lectura actual acabi amb els caràcters que hi hagi en aquell moment, encara que no n'hi hagi cap:

```
while ( (n=read(0, &car, 1)) > 0 )
        write(1, &car, 1);
```

    - Això, per convenció, s'interpreta com a final de fitxer (EOF)
  - Escriptura
    - Escriu un bloc de caràcters
      - Pot esperar que s'escrigui el CR per ser visualitzat per pantalla
    - El procés no es bloqueja
  - Aquest comportament bloquejant pot ser modificat mitjançant crides al sistema que canvien el comportament dels dispositius

Característiques pipes



- **Pipe**
  - Lectura
    - Si hi ha dades, agafa les que necessita (o les que hi hagi)
    - Si no hi ha dades, es queda bloquejat fins que n'hi hagi alguna
    - Si la *pipe* està buida i no hi ha cap possible escriptor (tots els canals d'escriptura de la *pipe* tancats), el procés lector rep *EOF*
      - Cal tancar sempre els canals que no es facin servir
  - Escriptura
    - Si hi ha espai a la *pipe*, escriu les que té (o les que pugui)
    - Si la *pipe* està plena, el procés es bloqueja
    - Si no hi ha cap possible lector, rep *SIGPIPE*
  - Aquest comportament bloquejant pot ser modificat mitjançant crides al sistema que canvien el comportament dels dispositius (fcntl)

Crides a sistema: espai de noms i permisos



Servicio	Llamada a sistema
Crear / eliminar enllaç a fitxer/soft-link	link / unlink/symlink
Canviar permisos de un fitxer	chmod
Canviiar propietari/grup d'un fitxer	chown / chgrp
Obtenir informació de l'inode	stat, lstat, fstat

- Existeixen més crides a sistema que ens permeten manipular:
  - Permisos
  - Vincles
  - Característiques
  - etc

EXEMPLES



Accés byte a byte

- Lectura per l'entrada estàndard i escriptura per la sortida estàndard



```
while ((n = read(0, &c, 1)) > 0)
    write(1, &c, 1);
```

  - Observacions:
    - Generalment, els canals estàndard ja estan oberts per defecte
    - Es llegeix fins que no queden dades (`n==0`), que depèn del tipus de dispositiu
    - El total de crides a sistema depèn de quants bytes llegir
    - Els canals 0 i 1 poden estar vinculats amb qualsevol dispositiu lògic que permeti llegir i escriure

```
#exemple1 →                    entrada=terminal, sortida=terminal
#exemple1 <disp1 →             entrada=disp1, sortida=terminal
#exemple1 <disp1  >disp2 →     entrada=disp1, sortida=disp2
```

Accés amb buffer d'usuari

- Igual, però llegim blocs de bytes (chars en aquest cas)

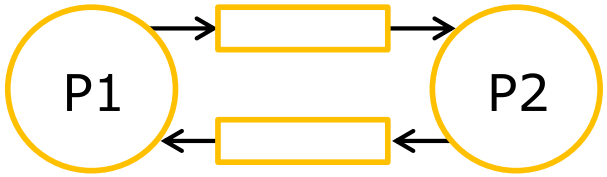


```
char buf[SIZE];
...
while ((n = read(0, buf, SIZE)) > 0)
    write(1, buf, n);
```

  - Observacions:
    - Compte! Cal escriure `n` bytes
      - Demanem SIZE bytes, ja que la variable ho permet, però és un màxim, pot ser que haguem llegit menys
    - Quines diferències hi ha en quant a rendiment? Quantes crides a sistema farem?

Intercanvi d'informació amb pipes

- Crear un esquema de procesos que sigui equivalent a aquest



- 2 pipes
- P1 envia per pipe1 i rep de pipe2
- P2 al revés

```
void p1(int fdin,int fdout);
void p2(int fdin,int fdout);
```

```
1.  int pipe1[2], pipe2[2],pidp1,pidp2;
2.  pipe(pipe1);
3.  pipe(pipe2);
4.  pidp1=fork();
5.  if (pidp1==0){
6.        close(pipe1[0]);
7.        close(pipe2[1]);
8.        p1(pipe2[0],pipe1[1]);
9.        exit(0);
10. }
11. close(pipe1[1]);
12. close(pipe2[0]);
13. pidp2=fork();
14. if (pidp2==0){
15.       p2(pipe1[0],pipe2[1]);
16.       exit(0);
17. }
18. close(pipe1[0]);close(pipe2[1]);
19. while(waitpid(-1,null,0)>0);
```

Accés aleatori i càlcul de la mida

- Qué fa el següent fragment de codi?



```
fd = open("abc.txt", O_RDONLY);
while (read(fd, &c, 1) > 0) {
    write(1, &c, 1);
    lseek(fd, 4, SEEK_CUR);
}
```

Trobareu aquest codi a : ejemplo1.c

- Y aquest altre?



```
fd = open("abc.txt", O_RDONLY);
size = lseek(fd, 0, SEEK_END);
printf("%d\n", size);
```

Trobareu aquest codi a : ejemplo2.c

pipes i bloquejos



```
int fd[2];
...
pipe(fd);
pid = fork();
if (pid == 0) {                                  // Codi del fill
    while (read(0, &c, 1) > 0) {
        // Llegeix la dada, la processa i l'envia
        write(fd[1], &c, 1);
    }
}
else {                                           // Codi del pare
    while (read(fd[0], &c, 1) > 0) {
        // Rep la dada, la processa i la escriu
        write(1, &c, 1);
    }
}
...
```

Teniu disponible aquest codi a: pipe_basic.c

- Compte! El pare ha de tancar `fd[1]` si no es vol quedar bloquejat!

Compartir el punter de l/e

- Què fa aquest fragment de codi?



```
...
fd = open("fitxer.txt", O_RDONLY);
pid = fork();
while ((n = read(fd, &car, 1)) > 0 )
       if (car == 'A') numA++;
sprintf(str, "El número d'As és %d\n", numA);
write(1, str, strlen(str));
...
```

Teniu disponible aquest codi a: exemple1.c

No compatir el punter de l/e

- Què fa aquest fragment de codi?



```
...
pid = fork();
fd = open("fitxer.txt", O_RDONLY);
while ((n = read(fd, &car, 1)) > 0 )
       if (car == 'A') numA++;
sprintf(str, "El número d'As és %d\n", numA);
write(1, str, strlen(str));
...
```

Teniu disponible aquest codi a: exemple2.c

Redirecció entrada i sortida std

- Què fa aquest fragment de codi?



```
...
pid = fork();
if ( pid == 0 ) {
    close(0);
    fd1 = open("/dev/disp1", O_RDONLY);
    close(1);
    fd2 = open("/dev/disp2", O_WRONLY);
    execv("programa", "programa", (char *)NULL);
}
...
```

Pipes y redirecció



```
...
pipe(fd);
pid1 = fork();
if ( pid1 != 0 ) {                              // PARE
    pid2 = fork();
    if ( pid2 != 0 ) {                          // PARE
        close(fd[0]); close(fd[1]);
        while (1);
    }
    else {                                      // FILL 2
        close(0); dup(fd[0]);
        close(fd[0]); close(fd[1]);
        execlp("programa2", "programa2", NULL);
    }
}
else {                                          // FILL 1
        close(1); dup(fd[1]);
        close(fd[0]); close(fd[1]);
        execlp("programa1", "programa1", NULL);
}
```

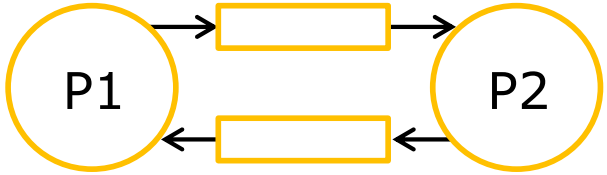
Mmap



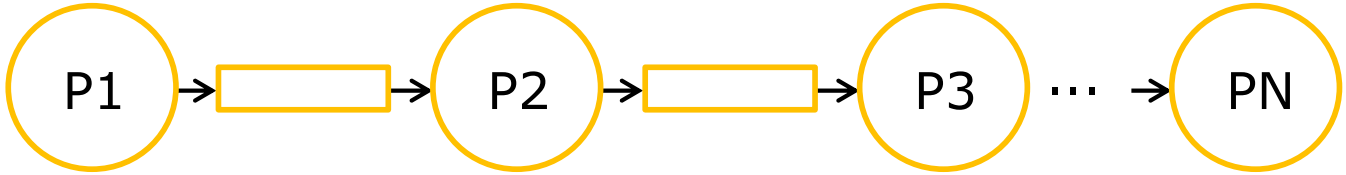
```
...
int count_As(const char *start, size_t length) {
    int count = 0;
    for (size_t i = 0; i < length; i++) {
        if (start[i] == 'A') count++;
    }
    return count;
}
int fd = open("fitxer.txt", O_RDONLY);
struct stat sb; fstat(fd, &sb);
char *addr = mmap(NULL, sb.st_size, PROT_READ, MAP_PRIVATE, fd, 0); close(fd);
if (fork>0) {
    size_t half = file_size / 2;
    int parent_count = count_As(file_data, half);
}
else {
    size_t half = file_size / 2;
    int count = count_As(file_data + half, file_size - half);
    printf("Fill: %d 'A's trobades\n", count);
    exit(count); // si count <256
}
int status;
waitpid(pid, &status, 0);
int child_count = WEXITSTATUS(status);
// Sumem els resultats
int total = parent_count + child_count;
printf("Pare: %d 'A's trobades\n", parent_count);
printf("Total: %d 'A's\n", total);
// Alliberem la memòria mapejada
munmap(file_data, file_size);
```

Exercicis a classe (1) (Col.P clase)

- Escriviu un fragment de codi que creï dos processos *p1* i *p2*, i els connecti mitjançant dues *pipes* pels canals d'E/S estàndard: en la primera *p1* hi escriurà i *p2* hi llegirà, en la segona *p2* hi escriurà i *p1* hi llegirà

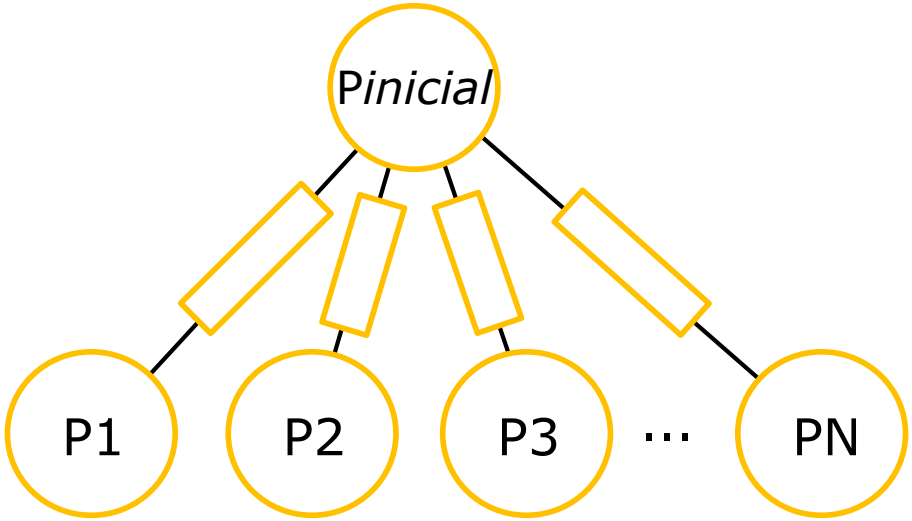


- Escriviu un fragment de codi que creï una seqüència de *N* processos en cadena: cada procés *p_i* crea un sol descendent *p_i+1*, fins arribar a *p_N*. Cada procés s'ha de comunicar en cadena amb l'ascendent i el descendent mitjançant una *pipe* pel canal d'E/S estàndard, de manera que el que escrigui el primer procés pugui ésser enviat per cadena fins al darrer procés



Exercicis a classe (2)

- Escriviu un fragment de codi que creï *N* processos en seqüència: el procés inicial crea tots els descendents *p_1* fins arribar a *p_N*. Cada procés s'ha de comunicar amb el procés pare mitjançant una *pipe*; el pare ha de poder escriure a totes les *pipes* (a través dels canals 3..N+2) i els fills han de llegir de la *pipe* per la seva entrada estàndard.



RELACIÓ ENTRE CRIDES A SISTEMA / ESTRUCTURES DE DADES



Open



- Localitza l'inode del fitxer amb el que es vol treballar i el deixa a memòria
  - Quin és l'inode? Cal llegir el directori del fitxer. Dos situacions:
    - O el directori està a la cache de directoris i simplement s'accedeix
    - O el directori està a disc i s'ha de llegir (i es deixa a memòria).
      - Com sabem quins blocs llegir? → estan a l'inode
      - Quin es l'inode? → està al directori (repetim el procés)
- Si localitza l'inode
  - Comprova si l'accés es correcte (els permisos d'accés són els adequats) → si no retorna errro
  - Si és un soft-link, llegeix el path del fitxer al que apunta i localitza l'inode corresponem (repetim el procés)
    - Si el path es curt, es troba al mateix inode
    - Si el path es llarg, es guarda en un bloc de dades separat
  - Modifica la taula de canals, la taula de fitxers oberts i la taula d'inodes
- Si no localitza l'inode: error per accedir a un fitxer que no existeix
- No accedeix a cap bloc de dades del fitxer objectiu

Open (cont)



- Si utilitzem l'open per a crear un nou fitxer
  - S'ha de reservar i inicialitzar un nou inode
    - Actualitzar llista d'inodes lliures al superbloc
    - Actualitzar directori a on es crea el fitxer
      - modificar el bloc de dades del directori
      - modificar l'inode amb la nova mida del directori

Read



- Tenim, a la taula d'inodes, l'inode corresponent (i la informació sobre els blocs de dades)
- Tenim a la TFO el valor del punter de lectura/escriptura (la posició del fitxer que toca accedir)
- Cal calcular els blocs involucrats:
  - Primer comprovem si estem al final del fitxer (valor del punter == mida del fitxer)
  - Després calculem els blocs
    - Dividint el valor del punter entre la mida de bloc obtenim l'index del primer bloc a llegir
    - Amb el paràmetre mida de la syscall podem calcular quants blocs més cal llegir
- Si els blocs ja s'havien utilitzat abans els podem trobar a memòria, si no cal llegir-los de disc (i es deixen a memòria).

Mmap

- Igual que en read, el kernel consulta l'**inode** del fitxer per obtenir metadades com: mida, permisos, punters a blocs
- Diferència amb el read: no s'accedeix a les dades, només es crea la regió a l'espai d'adreces i es vincula amb el contingut del fitxer
- Quan el procés accedeixi a les adreces de la regió es farà la lectura real:
  - **Càrrega sota demanda**
    - Primer accés a una pàgina de la regió provoca page fault que el sistema resol carregant de disc (si és necessari) el bloc involucrat

Write



- La diferència amb el read es que si l'escriptura es fa al final del fitxer pot ser necessari afegir més blocs al fitxer. En aquest cas, caldrà modificar la llista de blocs lliures del superbloc i actualitzar l'inode del fitxer amb els nous blocs i la nova mida del fitxer

Close



- Provoca l'actualització de l'inode amb els canvis que s'hagin fet (com a mínim la data d'últim accés)